Chapter 12 — Versioning benefits and challenges

Tools only help if teams understand why versioning matters and where it fails

Learning objectives

- By the end of this part

Benefits at platform scale

- Reproducibility and audit benefits of versioning are developed in Chapters 8 and 13
- At scale, the distinctive gains are different

Example 12.18 — Rollback After Corrupt Preprocessing

- Example 12.18 — hands-on module
- Example 12.18 shows why rollback matters when a transform corrupts shared data
- If a preprocessing job corrupts a shared feature table
- Explore the chapter example module
- View files: `modules/chapter12/example18/`

Challenge: storage overhead

- General versioning discipline
- At lakehouse and object-store scale, storage overhead dominates
- Full copies of petabyte tables are rarely affordable
- Delta and snapshot formats keep costs manageable by storing diffs

Challenge: schema evolution under concurrency

- Adding columns or changing types while readers and writers share a table requires
- Lakehouse formats such as Delta Lake, Hudi

Discipline still matters

- Agreement on when a schema break is a new table
- Versioning succeeds when engineering practice and lakehouse primitives reinforce each

Takeaways

- At scale, versioning enables snapshot pinning, partition rollback
- Example 12.18 shows rollback after corrupt preprocessing instead of a full rebuild
- Storage overhead and concurrent schema change remain the main challenges
- Chapter 8 still governs team hygiene; Chapter 12 governs lakehouse cost and concurrency

Next

- Complete the quiz for this part
- The next part turns from what changed to how data moved